

CI/CD — The Complete Guide

Jenkins • GitHub Actions • GitLab Runners • Azure Pipelines • AWS CI/CD

For Java Full Stack Developers (Spring Boot Backend + React Frontend)

Before CI/CD → Concepts → Pipelines → Real Scripts → Deployment → Interview
Q&A

A practical, simple-language reference for interview preparation and daily real-world use

Table of Contents

1. Before CI/CD — Real Problems in Organizations
2. What is CI/CD and What Problems It Solves
3. CI vs CD vs Continuous Deployment — Clear Difference
4. The CI/CD Toolscape — Jenkins, GitHub Actions, GitLab CI, Azure Pipelines, AWS CI/CD
5. Where Git and Maven Fit Into CI/CD
6. Anatomy of a Pipeline — Stages Explained
7. Prerequisites Before You Create Any Pipeline
8. Spring Boot Backend — Full CI/CD Example (End to End)
9. React Frontend — Full CI/CD Example (End to End)
10. Jenkins Deep Dive with Real Jenkinsfile
11. GitHub Actions Deep Dive with Real Workflow YAML
12. GitLab CI/CD Runners Deep Dive with Real .gitlab-ci.yml
13. Azure Pipelines Deep Dive with Real azure-pipelines.yml
14. AWS CI/CD Deep Dive (CodePipeline, CodeBuild, CodeDeploy, CodeCommit)
15. Tool vs Tool — Full Comparison Table
16. Deployment Strategies (Recreate, Rolling, Blue-Green, Canary)
17. Azure DevOps in Depth — Our Organization's Real Setup & Configuration
18. How Deployment Works (Dev → QA → Staging → Prod)
19. How Integration Testing Works in a Pipeline
20. Docker + Kubernetes in the CI/CD Flow
21. Secrets, Approvals and Environments
22. Common CI/CD Issues and How to Fix Them (End to End Troubleshooting)
23. CI/CD Best Practices for Full Stack Teams
24. CI/CD Cheat Sheet
25. Interview Questions & Answers (Basic → Advanced → Scenario Based)
26. Final Summary

1. Before CI/CD — Real Problems in Organizations

1.1 A Day Without CI/CD

Imagine a Java full stack team (Spring Boot + React) releasing an e-commerce app **without** any CI/CD pipeline:

- A developer finishes a feature, builds the project on their own laptop using `mvn clean install`, and manually copies the JAR to the server using FTP/WinSCP.
- "It works on my machine" — the build passes locally but fails on the server because of different Java versions, missing environment variables, or a forgotten config file.
- No one runs unit tests before deployment, so bugs reach production and are found by customers, not developers.
- Deploying the React frontend means manually running `npm run build` and copying the `dist` / `build` folder to a web server via SFTP — very slow and error-prone.
- Releases happen once a month because they are risky, manual, and require a "deployment day" with the whole team on a call, often at night.
- If something breaks in production, rollback means someone manually copying the previous JAR/zip back — if they can even find it.
- There is no consistent record of *what* was deployed, *when*, and *by whom*.

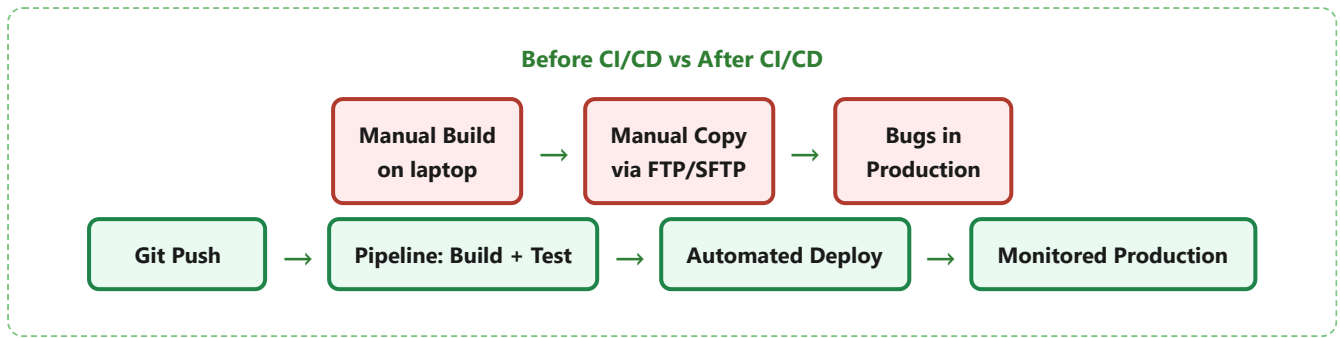
Real pain: A release engineer manually deploys the wrong build (an old JAR file) to production on a Friday evening. Nobody notices until Monday morning when customers report missing features. There is no audit trail to explain what happened.

1.2 Problems Manual Releases Create for Organizations

Problem	Impact on Organization
Manual build & deploy	Slow, inconsistent, human errors, different results on different machines
No automated testing before deploy	Bugs reach production, customer trust drops
No standard process	Every developer deploys differently; tribal knowledge, "only Raj knows how to deploy"
Slow release cycle	Business loses competitive speed; features take weeks to reach users
Hard rollback	Downtime lasts longer when something goes wrong
No traceability	Cannot answer "what code is running in production right now?"
Fear of deployment	Team avoids releasing frequently because every release is risky

1.3 What Organizations Needed

- An automatic way to build and test code the moment it is pushed to Git.
- A repeatable, scripted deployment process — same steps every time, no human memory needed.
- Fast feedback: if a developer breaks the build, they should know within minutes, not days.
- Ability to release many times a day safely, with approvals only where needed (like production).
- A clear history: which commit, which build, which environment, deployed by which pipeline run.



2. What is CI/CD and What Problems It Solves

2.1 Simple Definition

CI/CD stands for **Continuous Integration** and **Continuous Delivery/Deployment**. It is an automated pipeline (a series of scripted steps) that takes your code from a Git repository, builds it, tests it, packages it, and deploys it to servers — automatically, every time code changes, without manual steps.

2.2 Continuous Integration (CI)

Every time a developer pushes code (or raises a Pull Request), the pipeline automatically:

1. Pulls the latest code from Git
2. Builds it (e.g., `mvn compile` for Java, `npm install & build` for React)
3. Runs unit tests and code quality checks
4. Reports success/failure back to the developer within minutes

Goal: catch integration bugs early, before they reach other developers or production.

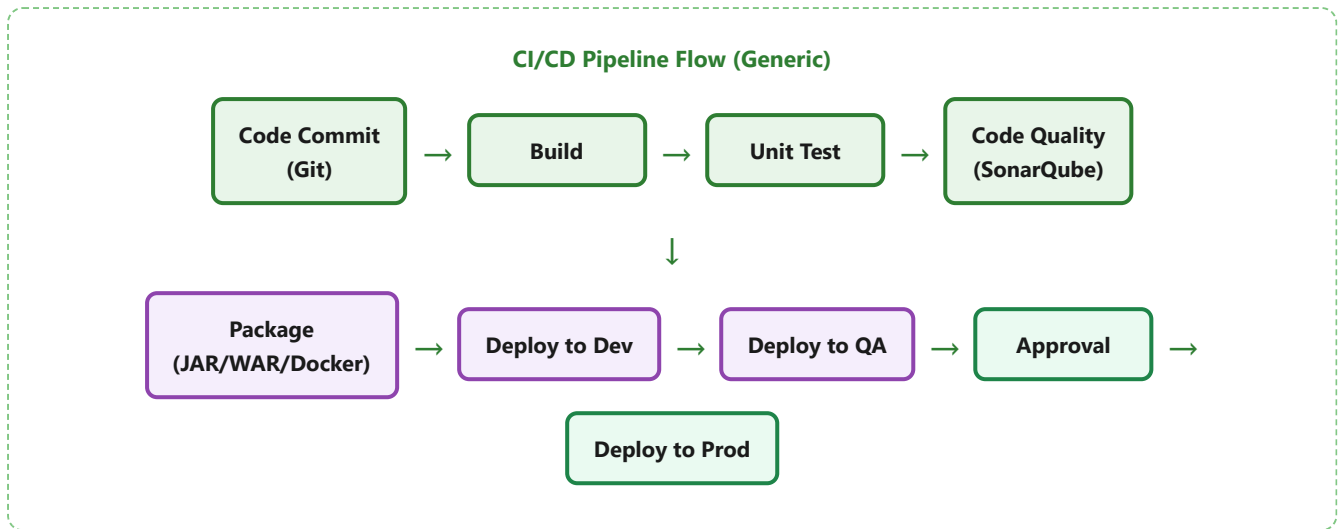
2.3 Continuous Delivery vs Continuous Deployment

Term	Meaning	Production Deploy
Continuous Delivery	Code is always in a deployable state; the final deploy-to-production step needs a manual approval/click	Manual trigger (button click)
Continuous Deployment	Every change that passes all tests is deployed to production automatically — no human click	Fully automatic

Real world: Most banking/finance orgs use Continuous **Delivery** (manual approval before prod) for compliance reasons. Product companies like fast-moving startups often use Continuous **Deployment** (fully automatic) for low-risk services.

2.4 What Problems CI/CD Solves

Old Problem	CI/CD Solution
"Works on my machine"	Build always happens in a clean, identical pipeline environment (often a Docker container)
Bugs found late	Automated tests run on every commit — fail fast
Manual, risky deploys	Same scripted deployment steps every single time
Slow releases	Pipelines run in minutes; teams can release multiple times a day
No rollback plan	Redeploy the previous successful pipeline artifact/version instantly
No audit trail	Every pipeline run is logged: who pushed, what commit, what tests passed, who approved



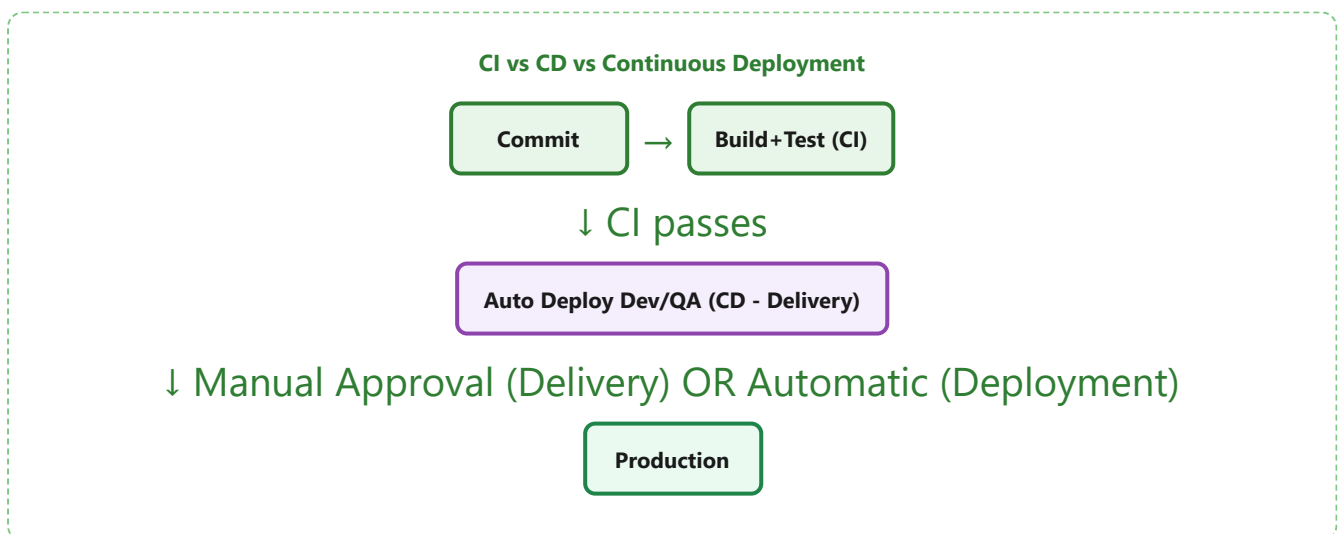
3. CI vs CD vs Continuous Deployment — Clear Difference

3.1 Side by Side

Stage	What Happens	Tools Involved	Triggered By
Continuous Integration	Build + Unit Test + Static Analysis on every push/PR	Jenkins, GitHub Actions, GitLab CI, Maven	Git push / PR opened
Continuous Delivery	Package + auto-deploy to Dev/QA/Staging; Prod needs manual click	Same tools + deployment scripts, Ansible, Helm	Successful CI + manual approval
Continuous Deployment	Fully automatic deploy to Production too, with no human step	Same + strong automated test coverage	Successful CI, automatically

3.2 Why the Distinction Matters in Real Organizations

- Regulated industries (banking, healthcare, government) almost always use **Continuous Delivery** — a human must approve the production release for compliance/audit reasons.
- SaaS/product companies with strong automated test suites (e.g., feature-flag driven releases) often use **Continuous Deployment** for internal services and some customer-facing services.
- In interviews, always mention this real distinction — most companies use CI + CD (delivery), not full continuous deployment, because production still needs a gate/approval.



4. The CI/CD Toolscape

4.1 The Big Players

Tool	Owner/Type	Where Pipeline is Defined	Best Fit
Jenkins	Open-source, self-hosted server	<code>Jenkinsfile</code> (Groovy-based Pipeline DSL)	Large enterprises needing full control, on-prem, plugins for everything
GitHub Actions	Built into GitHub	<code>.github/workflows/*.yaml</code>	Teams already using GitHub; fast setup, huge marketplace of actions
GitLab CI/CD (Runners)	Built into GitLab	<code>.gitlab-ci.yaml</code>	Teams using GitLab; tightly integrated with Merge Requests and Container Registry
Azure Pipelines	Microsoft Azure DevOps	<code>azure-pipelines.yaml</code>	Enterprises on Microsoft/Azure stack, .NET + Java mixed shops
AWS CI/CD	AWS (CodeCommit, CodeBuild, CodeDeploy, CodePipeline)	<code>buildspec.yaml</code> + Pipeline JSON/console/CDK	Teams fully hosted on AWS infrastructure

4.2 Common Thread Across All Tools

Even though syntax differs, every tool follows the same mental model:

1. **Trigger** — something starts the pipeline (push, PR, tag, schedule, manual)
2. **Agent/Runner** — a machine (VM, container, or cloud-hosted) that executes the steps
3. **Stages/Jobs** — logical groups of steps (Build, Test, Package, Deploy)
4. **Steps/Tasks** — individual commands (e.g., `mvn test`, `docker build`)
5. **Artifacts** — the output of the build (JAR, Docker image, static files) passed between stages
6. **Environments** — Dev, QA, Staging, Prod, each with its own config/secrets

If you understand this generic model, you can pick up any CI/CD tool quickly — only the YAML/DSL syntax changes.

5. Where Git and Maven Fit Into CI/CD

5.1 Git's Role

- Git is the **source of truth** — every pipeline starts by checking out a specific commit/branch from Git.
- Pipelines are triggered by Git events: push to `main` / `develop`, a Pull/Merge Request, or a new Git tag (e.g., `v1.2.0` for a release).
- Branching strategy directly drives pipeline behavior — e.g., pushing to `feature/*` branches might only run CI (build+test), while merging into `main` triggers full CD to production.

```
# Typical trigger rules seen in real pipelines
push to feature/*    -> run build + unit tests only
pull_request to develop -> run build + tests + code quality gate
push/merge to main   -> run build + tests + package + deploy to Dev/QA
tag v*.*.*           -> deploy to Production
```

5.2 Maven's Role (Java Projects)

Maven is the build tool for Spring Boot projects. Every Java CI/CD pipeline calls Maven commands as its "build" and "test" steps.

Maven Command	What It Does in the Pipeline
<code>mvn clean</code>	Removes old <code>target/</code> build output
<code>mvn compile</code>	Compiles Java source code
<code>mvn test</code>	Runs JUnit/Mockito unit tests
<code>mvn package</code>	Builds the deployable JAR/WAR file
<code>mvn verify</code>	Runs integration tests (Failsafe plugin) after packaging
<code>mvn install</code>	Installs the artifact into the local <code>.m2</code> repo (used for multi-module projects)
<code>mvn deploy</code>	Uploads the built artifact to a remote repository (Nexus/Artifactory)

```
mvn clean package -DskipTests=false
```

Real pipelines usually run `mvn clean verify` which does compile → test → package → integration-test in one command, then archive the resulting JAR as a build artifact.

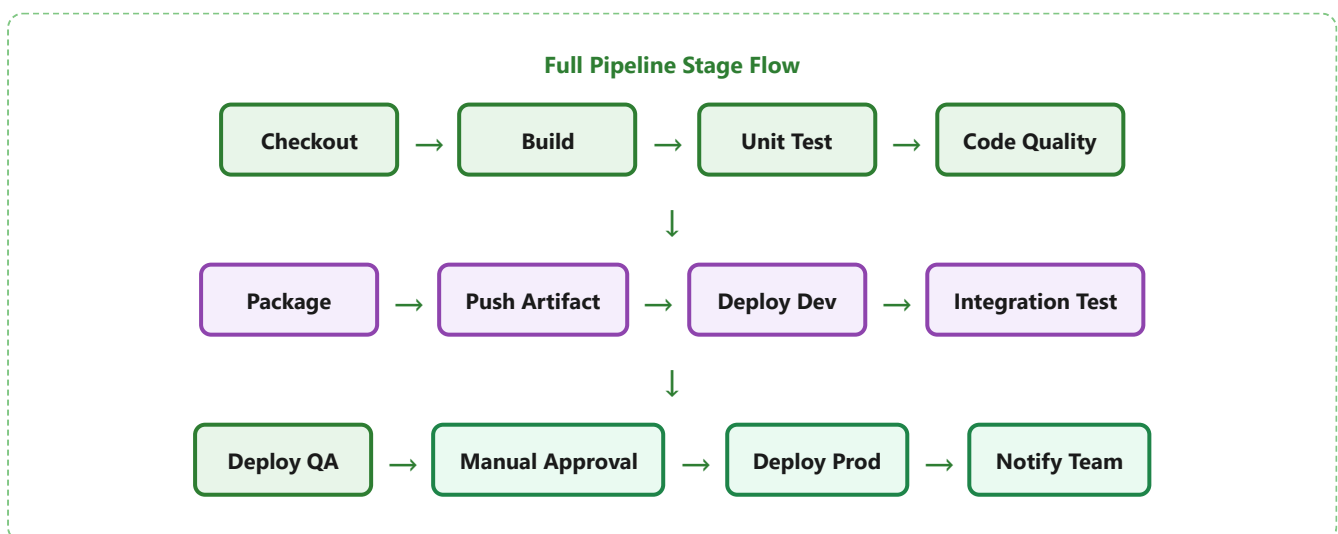
5.3 Dependency Management in CI

- Maven downloads dependencies from `pom.xml` — in CI, the `.m2` cache is often cached between runs to speed up builds (avoids re-downloading every time).
- Private/internal libraries are usually pulled from a company **Nexus** or **Artifactory** repository, configured via `settings.xml` with credentials stored as pipeline secrets.
- For React, the equivalent is `package.json` + `node_modules`, cached using the `npm` / `yarn` lock file hash as a cache key.

6. Anatomy of a Pipeline — Stages Explained

6.1 Standard Stages Used Across All Tools

Stage	Purpose	Typical Commands
Checkout	Pull code from Git at the triggering commit	<code>git clone</code> / <code>checkout</code> (done automatically by most tools)
Build	Compile the source code	<code>mvn compile</code> , <code>npm install</code>
Unit Test	Run fast automated tests, no external dependencies	<code>mvn test</code> , <code>npm test</code>
Code Quality	Static analysis, coverage, security scan	SonarQube, ESLint, Checkstyle
Package	Create deployable artifact	<code>mvn package</code> , <code>npm run build</code> , <code>docker build</code>
Push Artifact	Store the artifact somewhere reusable	Push JAR to Nexus, push image to Docker Hub/ECR/ACR/GitLab Registry
Deploy - Dev	Automatic deploy to a shared dev environment	<code>kubectl apply</code> , <code>ansible-playbook</code> , SSH+scp
Integration/E2E Test	Test the deployed app end-to-end (API tests, Selenium/Cypress)	Postman/Newman, Cypress, RestAssured
Deploy - QA/Staging	Deploy to a pre-prod environment for manual/exploratory testing	Same deploy scripts, different config
Approval Gate	Manual sign-off before production (Continuous Delivery)	Built-in "Environments/Approvals" feature of the tool
Deploy - Prod	Final release to live users	Blue-green / rolling / canary deployment scripts
Notify	Tell the team the result	Slack/Teams/Email notification step



7. Prerequisites Before You Create Any Pipeline

7.1 Things You Must Have Ready

1. **Git repository** with the project already pushed (GitHub/GitLab/Bitbucket/Azure Repos).
2. **Working local build** — the project must build successfully locally first: `mvn clean package` for Spring Boot, `npm install & npm run build` for React. If it does not build locally, it will not build in the pipeline.
3. **Dockerfile** (if deploying as containers) — defines how to package the app as an image.
4. **Target environment access** — server/VM/Kubernetes cluster/cloud account credentials where the app will be deployed.
5. **Secrets/credentials** stored securely in the CI/CD tool's secret store (never hardcoded): DB passwords, cloud keys, registry logins.
6. **Build agent/runner** — either self-hosted (a Jenkins agent VM, a GitLab Runner machine) or cloud-hosted (GitHub-hosted runners, Azure-hosted agents).
7. **Branching strategy decided** — which branch maps to which environment (e.g., `develop` → Dev, `release/*` → QA, `main` → Prod).
8. **Test suite in place** — unit tests (JUnit/Jest) at minimum; integration/E2E tests are a bonus.

Common mistake: Teams try to write a pipeline before the project even builds cleanly on their own laptop. Always get a clean local build + tests passing first — the pipeline should just automate what already works manually.

7.2 Checklist Before Writing YAML/Jenkinsfile

Checklist Item	Why It Matters
pom.xml / package.json is correct and builds	Pipeline literally runs these build tools
Java version / Node version fixed	Avoids "works on my machine" version drift
.gitignore excludes target/, node_modules/, .env	Keeps repo clean; secrets not committed
Environment-specific config externalized	application-dev.yml, application-prod.yml, or env vars — not hardcoded
Access to deployment target confirmed	SSH key, kubeconfig, or cloud IAM role ready before writing deploy stage

8. Spring Boot Backend — Full CI/CD Example (End to End)

8.1 The Real-Time Scenario

We have an **Order Service** — a Spring Boot REST API (Java 17, Maven, connects to MySQL) — stored in Git. We want: every push to `develop` builds+tests+deploys to Dev automatically; merging to `main` deploys to Production after manual approval.

8.2 Project Structure

```
order-service/  
├─ src/main/java/com/shop/order/...  
├─ src/test/java/com/shop/order/...  
├─ pom.xml  
├─ Dockerfile  
├─ Jenkinsfile  
├─ .github/workflows/backend-ci-cd.yml  
└─ .gitlab-ci.yml
```

8.3 Step 1 — Confirm Local Build Works

```
mvn clean verify  
# BUILD SUCCESS -> now safe to automate in a pipeline
```

8.4 Step 2 — Dockerfile for the Spring Boot App

```
# Multi-stage build: compile with Maven, run with a small JRE image  
FROM maven:3.9-eclipse-temurin-17 AS build  
WORKDIR /app  
COPY pom.xml .  
RUN mvn dependency:go-offline -B  
COPY src ./src  
RUN mvn clean package -DskipTests  
  
FROM eclipse-temurin:17-jre-alpine  
WORKDIR /app  
COPY --from=build /app/target/order-service-*.jar app.jar  
EXPOSE 8080  
ENTRYPOINT ["java","-jar","app.jar"]
```

8.5 Step 3 — What the Pipeline Does (Any Tool)

1. Checkout code from Git (branch that triggered the run)
2. Set up Java 17 + Maven cache
3. `mvn clean verify` — compiles, runs unit tests, runs integration tests
4. Publish test reports (JUnit XML) so failures show in the pipeline UI
5. `docker build` the image and tag it with the Git commit SHA (e.g., `order-service:a1b2c3d`)
6. Push the image to a registry (Docker Hub / ECR / ACR / GitLab Container Registry)

7. Deploy to Dev automatically (e.g., `kubectl set image` or `docker-compose pull & up`)
8. Run a smoke/integration test against Dev (hit `/actuator/health`)
9. Wait for manual approval → deploy the same image to Production

Key real-world rule: Build the artifact/Docker image **once**, then promote the exact same image through Dev → QA → Prod. Never rebuild per environment — that risks "it was a different build" bugs.

8.6 Application Config per Environment

```
# application-dev.yml
spring.datasource.url: jdbc:mysql://dev-db:3306/orders
# application-prod.yml
spring.datasource.url: ${DB_URL}      # injected as env var / secret by the pipeline
```

8.7 Deployment Mechanics for Spring Boot

Target	How Deploy Happens
Plain VM	SSH into server, <code>scp</code> the JAR, restart via <code>systemctl restart order-service</code>
Docker host	SSH + <code>docker pull && docker-compose up -d</code>
Kubernetes	<code>kubectl set image deployment/order-service order-service=registry/order-service:sha</code> (rolling update)
AWS Elastic Beanstalk / ECS	CodeDeploy or ECS service update triggers a new task with the new image
Azure App Service	Azure Pipelines "Deploy to App Service" task pushes the JAR/container directly

9. React Frontend — Full CI/CD Example (End to End)

9.1 The Real-Time Scenario

The React app (Shop UI) is a completely separate Git repository/pipeline from the Spring Boot backend — frontend and backend are deployed independently, which is normal in real organizations (separate release cadence, separate teams).

9.2 Project Structure

```
shop-ui/  
├─ src/  
├─ public/  
├─ package.json  
├─ Dockerfile  
├─ nginx.conf  
└─ .github/workflows/frontend-ci-cd.yml
```

9.3 Step 1 — Confirm Local Build Works

```
npm install  
npm run test -- --watchAll=false  
npm run build  
# creates the production-ready static build/ folder
```

9.4 Step 2 — Dockerfile for React (served via Nginx)

```
# Stage 1: build the static files  
FROM node:20-alpine AS build  
WORKDIR /app  
COPY package*.json ./  
RUN npm ci  
COPY . .  
RUN npm run build  
  
# Stage 2: serve with nginx  
FROM nginx:alpine  
COPY --from=build /app/build /usr/share/nginx/html  
COPY nginx.conf /etc/nginx/conf.d/default.conf  
EXPOSE 80
```

9.5 Step 3 — What the Pipeline Does

1. Checkout code
2. Set up Node.js (cache `node_modules` using `package-lock.json` hash)
3. `npm ci` (clean, reproducible install — always used in CI instead of `npm install`)
4. `npm test` — runs Jest/React Testing Library unit tests
5. `npm run lint` — ESLint code quality check

6. `npm run build` — produces optimized static files
7. Build a Docker image (Nginx serving the static files) OR upload static files directly to an S3 bucket / Azure Static Web App / Netlify
8. Deploy to Dev → run Cypress E2E tests against it → deploy to Prod (CDN cache invalidation if using CloudFront)

9.6 Frontend Talking to Backend Across Environments

```
// .env.development
REACT_APP_API_URL=http://dev-api.shop.com
// .env.production
REACT_APP_API_URL=https://api.shop.com
```

React reads the correct `REACT_APP_API_URL` at build time per environment, so the same code base produces environment-specific builds — this is why frontend and backend pipelines can run and deploy fully independently while still talking to the right API.

9.7 Static Hosting Options (Real World)

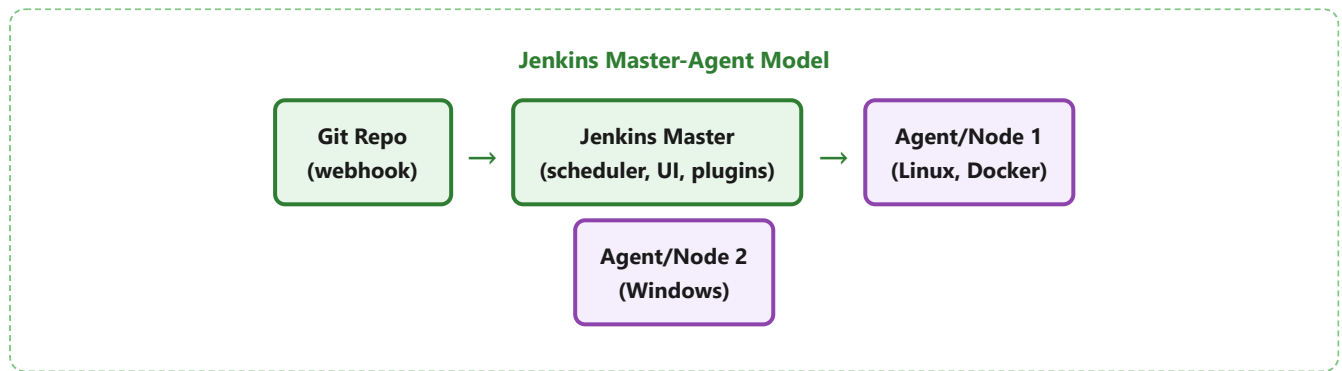
Option	Notes
Nginx/Docker on VM or K8s	Full control, common in enterprises with existing infra
AWS S3 + CloudFront	Very common for pure static React apps, cheap and fast (CDN)
Azure Static Web Apps	Native integration with Azure Pipelines/GitHub Actions
Netlify / Vercel	Popular for smaller teams/startups, zero-config CI/CD built in

10. Jenkins Deep Dive with Real Jenkinsfile

10.1 What Jenkins Is

Jenkins is a free, open-source, self-hosted automation server. You install it on your own VM/server; it has a huge plugin ecosystem (Git, Maven, Docker, Kubernetes, Slack, SonarQube, etc.). The pipeline is defined as code in a file called `Jenkinsfile`, written in Groovy-based syntax, checked into the same Git repo.

10.2 Jenkins Architecture



10.3 Real Jenkinsfile — Spring Boot Order Service

```
pipeline {
  agent any
  tools { maven 'Maven3'; jdk 'JDK17' }
  environment {
    IMAGE = "myorg/order-service:${env.GIT_COMMIT.take(7)}"
    REGISTRY_CREDS = credentials('docker-hub-creds')
  }
  stages {
    stage('Checkout') {
      steps { checkout scm }
    }
    stage('Build & Unit Test') {
      steps { sh 'mvn clean verify' }
      post {
        always { junit 'target/surefire-reports/*.xml' }
      }
    }
    stage('Code Quality') {
      steps { sh 'mvn sonar:sonar -Dsonar.host.url=$SONAR_URL' }
    }
    stage('Build Docker Image') {
      steps { sh "docker build -t $IMAGE ." }
    }
    stage('Push Image') {
      steps {
        sh "docker login -u $REGISTRY_CREDS_USR -p $REGISTRY_CREDS_PSW"
        sh "docker push $IMAGE"
      }
    }
    stage('Deploy to Dev') {
      steps { sh "kubectl set image deployment/order-service order-service=$IMAGE -n dev" }
    }
    stage('Approval') {
      steps { input message: 'Deploy to Production?', ok: 'Deploy' }
    }
    stage('Deploy to Prod') {
      steps { sh "kubectl set image deployment/order-service order-service=$IMAGE -n prod" }
    }
  }
  post {
    failure { slackSend channel: '#deployments', message: "Build FAILED: ${env.BUILD_URL}" }
    success { slackSend channel: '#deployments', message: "Deployed ${IMAGE} successfully" }
  }
}
```

10.4 Key Jenkins Concepts

Concept	Meaning
Master	The Jenkins server that schedules jobs and hosts the UI
Agent/Node	A machine that actually executes the pipeline steps
Declarative vs Scripted Pipeline	Declarative (shown above) is structured/simple; Scripted is full Groovy, more flexible but complex
Webhook	GitHub/GitLab calls Jenkins URL automatically on push, triggering the job

Credentials Store	Jenkins' built-in secret manager for passwords, tokens, SSH keys
-------------------	--

11. GitHub Actions Deep Dive with Real Workflow YAML

11.1 What GitHub Actions Is

GitHub Actions is CI/CD built directly into GitHub. No separate server to install — GitHub provides hosted runners (or you can register your own self-hosted runner). Pipelines are YAML files under `.github/workflows/`.

11.2 Real Workflow — Spring Boot Backend

```

name: Backend CI/CD
on:
  push:
    branches: [develop, main]
  pull_request:
    branches: [develop]

jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: 'temurin', java-version: '17', cache: 'maven' }
      - name: Build and Test
        run: mvn clean verify
      - name: Upload Test Report
        uses: actions/upload-artifact@v4
        with: { name: test-results, path: target/surefire-reports }

  docker-build-push:
    needs: build-test
    if: github.ref == 'refs/heads/develop' || github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build Image
        run: docker build -t myorg/order-service:${{ github.sha }} .
      - name: Login to Docker Hub
        run: echo "${{ secrets.DOCKERHUB_TOKEN }}" | docker login -u ${{ secrets.DOCKERHUB_USER }} --password-
stdin
      - name: Push Image
        run: docker push myorg/order-service:${{ github.sha }}

  deploy-dev:
    needs: docker-build-push
    if: github.ref == 'refs/heads/develop'
    runs-on: ubuntu-latest
    environment: dev
    steps:
      - name: Deploy to Dev
        run: kubectl set image deployment/order-service order-service=myorg/order-service:${{ github.sha }} -n
dev

  deploy-prod:
    needs: docker-build-push
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment:
      name: production # this environment has "Required reviewers" set in GitHub -> manual approval
    steps:
      - name: Deploy to Prod
        run: kubectl set image deployment/order-service order-service=myorg/order-service:${{ github.sha }} -n
prod

```

11.3 Real Workflow — React Frontend

```
name: Frontend CI/CD
on:
  push:
    branches: [main]

jobs:
  build-test-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: 'npm' }
      - run: npm ci
      - run: npm test -- --watchAll=false
      - run: npm run build
      - name: Deploy to S3
        run: aws s3 sync build/ s3://shop-ui-prod-bucket --delete
      - name: Invalidate CloudFront
        run: aws cloudfront create-invalidation --distribution-id $DIST_ID --paths "/*"
```

11.4 Key GitHub Actions Concepts

Concept	Meaning
Workflow	The whole YAML file — one automation triggered by an event
Job	A group of steps that runs on one runner; jobs can run in parallel or depend on each other (<code>needs: </code>)
Step / Action	A single task; "Actions" are reusable steps shared via the GitHub Marketplace (<code>uses: actions/checkout@v4</code>)
Secrets	Stored in repo/organization Settings → Secrets, referenced as <code>\${{ secrets.NAME }}</code>
Environments	Named deploy targets (dev/prod) that can require manual reviewer approval before running

12. GitLab CI/CD Runners Deep Dive with Real .gitlab-ci.yml

12.1 What GitLab CI/CD Is

GitLab has CI/CD built in, defined in a single `.gitlab-ci.yml` at the repo root. The actual execution happens on **GitLab Runners** — agents (shared, group, or project-specific) that pick up jobs and run them.

12.2 Real .gitlab-ci.yml — Spring Boot Backend

```
stages: [build, test, package, deploy-dev, deploy-prod]

variables:
  MAVEN_OPTS: "-Dmaven.repo.local=.m2/repository"

cache:
  paths: [.m2/repository]

build:
  stage: build
  image: maven:3.9-eclipse-temurin-17
  script: mvn compile

unit-test:
  stage: test
  image: maven:3.9-eclipse-temurin-17
  script: mvn test
  artifacts:
    when: always
    reports:
      junit: target/surefire-reports/TEST-*.xml

package-and-push:
  stage: package
  image: docker:24
  services: [docker:24-dind]
  script:
    - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA .
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
    - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA

deploy-dev:
  stage: deploy-dev
  script: kubectl set image deployment/order-service order-service=$CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA -n dev
  environment: { name: dev }
  rules:
    - if: '$CI_COMMIT_BRANCH == "develop"'

deploy-prod:
  stage: deploy-prod
  script: kubectl set image deployment/order-service order-service=$CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA -n prod
  environment: { name: production }
  when: manual # manual approval click in GitLab UI
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
```

12.3 Key GitLab CI/CD Concepts

Concept	Meaning
Stages	Ordered phases; all jobs in one stage run in parallel, next stage waits for it to finish
Runner	The agent (VM/Docker/K8s) that executes jobs; can be shared (GitLab-hosted) or self-hosted (project-specific)
when: manual	Turns a job into a manual approval gate — exactly like Jenkins' <code>input</code> step
CI/CD Variables	Secrets/config set in Settings → CI/CD → Variables, masked in logs
Built-in Container Registry	GitLab provides its own Docker registry per project — no separate Docker Hub needed

13. Azure Pipelines Deep Dive with Real azure-pipelines.yml

13.1 What Azure Pipelines Is

Part of Azure DevOps. Pipelines are defined in `azure-pipelines.yml`, executed on Microsoft-hosted agents or self-hosted agent pools. Very common in enterprises already using Azure Boards/Repos.

13.2 Real azure-pipelines.yml — Spring Boot Backend

```
trigger:
  branches: { include: [develop, main] }

pool: { vmImage: 'ubuntu-latest' }

stages:
- stage: BuildAndTest
  jobs:
  - job: Build
    steps:
    - task: JavaToolInstaller@0
      inputs: { versionSpec: '17', jdkArchitectureOption: 'x64', jdkSourceOption: 'PreInstalled' }
    - script: mvn clean verify
      displayName: 'Maven Build and Test'
    - task: PublishTestResults@2
      inputs: { testResultsFiles: '**/surefire-reports/*.xml' }

- stage: DockerPushDev
  condition: eq(variables['Build.SourceBranch'], 'refs/heads/develop')
  jobs:
  - job: BuildPushImage
    steps:
    - task: Docker@2
      inputs:
        containerRegistry: 'acrServiceConnection'
        repository: 'order-service'
        command: 'buildAndPush'
        tags: '$(Build.SourceVersion)'

- stage: DeployDev
  dependsOn: DockerPushDev
  jobs:
  - deployment: DeployToAKS
    environment: 'dev'
    strategy:
      runOnce:
        deploy:
          steps:
          - task: KubernetesManifest@0
            inputs: { action: 'deploy', manifests: 'k8s/deployment.yml' }

- stage: DeployProd
  condition: eq(variables['Build.SourceBranch'], 'refs/heads/main')
  jobs:
  - deployment: DeployToProdAKS
    environment: 'production' # configured with manual approvals + gates in Azure DevOps UI
    strategy:
      runOnce:
        deploy:
          steps:
          - task: KubernetesManifest@0
            inputs: { action: 'deploy', manifests: 'k8s/deployment.yml' }
```

13.3 Key Azure Pipelines Concepts

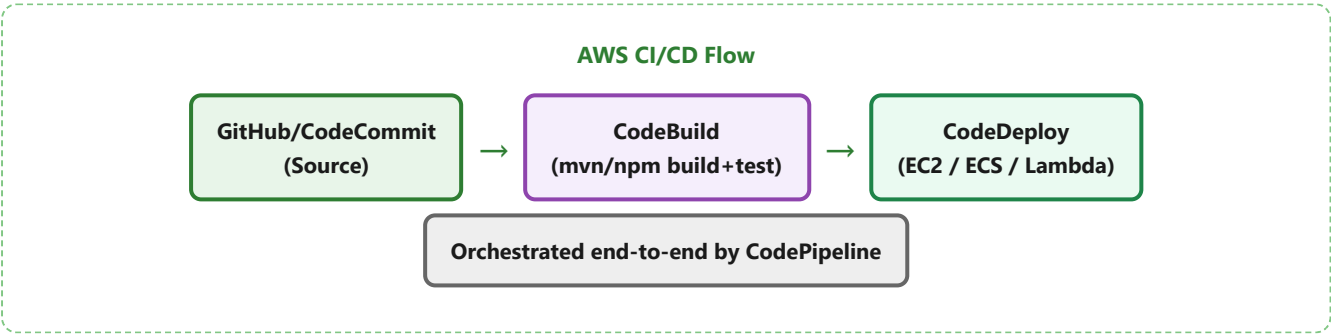
Concept	Meaning
---------	---------

Stage	Top-level phase (Build, Deploy Dev, Deploy Prod)
Job	A set of steps run on one agent within a stage
Task	A pre-built unit of work (like GitHub Actions) — huge marketplace: Docker@2, KubernetesManifest@0, AzureWebApp@1
Environments + Approvals	Configured in the Azure DevOps UI (not YAML) to require a specific person/group to approve before deploy
Service Connection	Secure link to external systems (ACR, AWS, K8s cluster) storing credentials centrally

14. AWS CI/CD Deep Dive (CodePipeline, CodeBuild, CodeDeploy, CodeCommit)

14.1 The AWS Native CI/CD Services

Service	Role
CodeCommit	AWS-hosted Git repository (rarely used now — most teams keep code on GitHub/GitLab/Bitbucket and just connect it)
CodeBuild	Runs the build/test commands, defined in <code>buildspec.yml</code> — equivalent to a Jenkins/GitHub Actions job
CodeDeploy	Handles the actual deployment to EC2, Lambda, or ECS, including blue-green deployments
CodePipeline	The orchestrator that chains Source → Build → Deploy stages together (like the overall Jenkinsfile/workflow)



14.2 Real buildspec.yml — Spring Boot Backend (CodeBuild)

```
version: 0.2
phases:
  install:
    runtime-versions: { java: corretto17 }
  pre_build:
    commands:
      - echo Logging in to ECR
      - aws ecr get-login-password | docker login --username AWS --password-stdin $ECR_URI
  build:
    commands:
      - mvn clean verify
      - docker build -t order-service:$CODEBUILD_RESOLVED_SOURCE_VERSION .
      - docker tag order-service:$CODEBUILD_RESOLVED_SOURCE_VERSION $ECR_URI:$CODEBUILD_RESOLVED_SOURCE_VERSION
  post_build:
    commands:
      - docker push $ECR_URI:$CODEBUILD_RESOLVED_SOURCE_VERSION
      - printf '["name":"order-service","imageUri":"%s"]' $ECR_URI:$CODEBUILD_RESOLVED_SOURCE_VERSION >
        imagedefinitions.json
  artifacts:
    files: [imagedefinitions.json]
  reports:
    unit-tests:
      files: ['target/surefire-reports/*.xml']
```

14.3 CodePipeline Stages (Console/CDK Definition, Conceptually)

```
Pipeline: order-service-pipeline
Stage 1 - Source: GitHub (webhook on push to main/develop)
Stage 2 - Build: CodeBuild runs buildspec.yml (test + docker build + push to ECR)
Stage 3 - DeployDev: CodeDeploy/ECS updates the "dev" ECS service with new image
Stage 4 - ManualApproval: SNS notification, someone clicks "Approve" in console
Stage 5 - DeployProd: CodeDeploy/ECS updates the "prod" ECS service (blue-green)
```

14.4 React Frontend on AWS

- CodeBuild runs `npm ci && npm test && npm run build`
- Build output pushed to an S3 bucket configured for static website hosting
- CloudFront serves it globally as a CDN; CodePipeline runs a CloudFront cache invalidation as the final deploy step

15. Tool vs Tool — Full Comparison Table

Feature	Jenkins	GitHub Actions	GitLab CI/CD	Azure Pipelines	AWS CI/CD
Hosting	Self-hosted (you manage the server)	GitHub-hosted or self-hosted runners	GitLab-hosted or self-hosted runners	Microsoft-hosted or self-hosted agents	Fully managed AWS services
Config Format	Jenkinsfile (Groovy DSL)	YAML workflow	YAML (.gitlab-ci.yml)	YAML (azure-pipelines.yml)	buildspec.yml + Pipeline definition (console/CDK/CFN)
Setup Effort	High — install/maintain server, plugins	Very low — built into GitHub	Low — built into GitLab	Medium — part of Azure DevOps org	Medium — needs several AWS services wired together
Best For	Full control, complex/legacy enterprise needs, on-prem	Teams already on GitHub, open-source projects	Teams already on GitLab, tight MR integration	Microsoft/.NET + Java enterprise shops	Teams fully invested in AWS infra
Secrets Management	Jenkins Credentials Store	GitHub Encrypted Secrets	GitLab CI/CD Variables (masked/protected)	Azure Key Vault / Variable Groups	AWS Secrets Manager / Parameter Store
Approval Gates	<code>input</code> step in Jenkinsfile	"Environments" with required reviewers	<code>when: manual</code> jobs	Environment approvals/checks (UI configured)	Manual Approval action in CodePipeline
Cost Model	Free, but you pay for server infra	Free minutes for public repos, paid minutes for private	Free tier minutes, paid for more	Free tier minutes, paid parallel jobs	Pay-per-build-minute + other AWS resource costs
Plugin/Extensibility	Massive plugin ecosystem (2000+)	Marketplace Actions (huge, community-driven)	Growing marketplace + native Docker/K8s integration	Marketplace Tasks (large, Microsoft-curated)	Integrates natively with all other AWS services

Interview tip: There is no single "best" tool — the right choice depends on where the code already lives (GitHub → GitHub Actions is natural) and what cloud/infra the company already uses (AWS shop → AWS CI/CD or Jenkins on EC2 is common; Azure shop → Azure Pipelines).

16. Deployment Strategies (Recreate, Rolling, Blue-Green, Canary)

16.1 Why Deployment Strategy Matters

Getting a build to "pass the pipeline" is only half the job. **How** the new version replaces the old one in production decides whether users notice downtime, whether a bad release can be undone in seconds, and how much extra infrastructure cost the org accepts. This is one of the most common real-world and interview topics.

16.2 Recreate Deployment

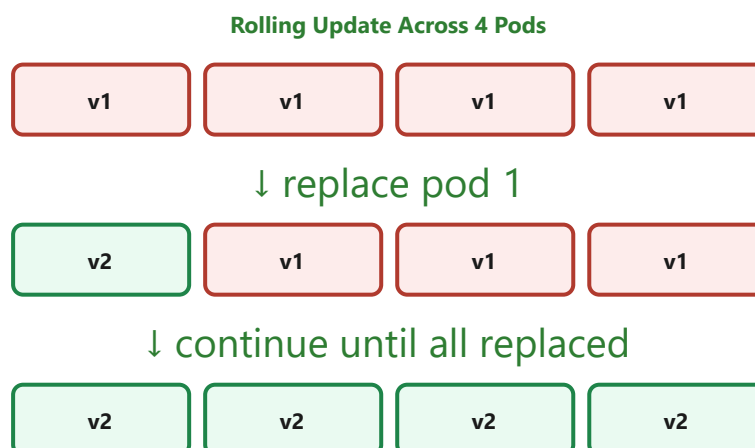
Stop the old version completely, then start the new version. Simple but causes downtime.



- **Used for:** internal tools, low-traffic apps, dev/QA environments.
- **Downside:** real downtime window — not acceptable for customer-facing production.

16.3 Rolling Deployment

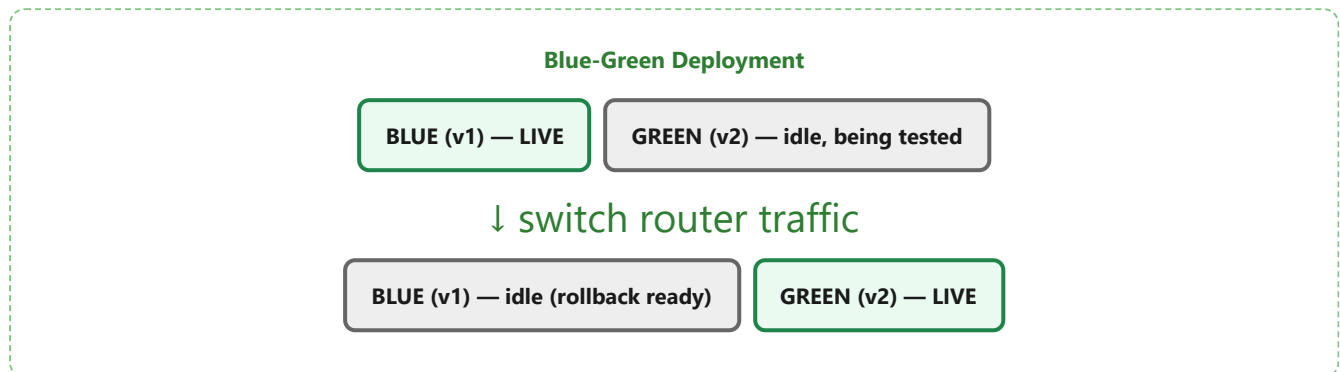
Replace instances/pods one (or a few) at a time. Some old and new versions run simultaneously during rollout. This is Kubernetes' default deployment strategy.



- **Used for:** most typical Spring Boot microservices on Kubernetes — zero downtime, gradual rollout.
- **Risk:** if v2 has a bug, some users hit v1 and others hit v2 at the same time (mixed versions) — the API contract must stay backward compatible during rollout.

16.4 Blue-Green Deployment

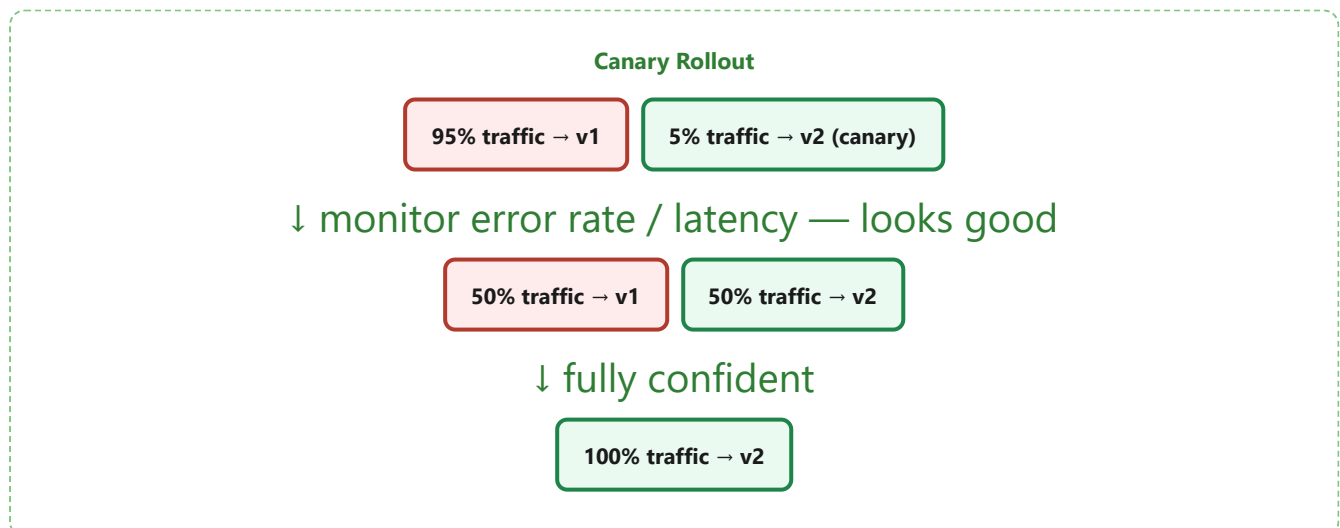
Keep two identical full environments — "Blue" (current live) and "Green" (new version). Deploy to Green, test it fully, then switch the router/load balancer to send all traffic to Green instantly. Blue stays idle as an instant rollback target.



- **Used for:** critical production releases where instant rollback matters (banking, e-commerce checkout).
- **Cost:** needs double the infrastructure momentarily (or two full environments always up).
- **Rollback:** just flip the router back to Blue — seconds, not a redeploy.

16.5 Canary Deployment

Release the new version to a small % of real users first (e.g., 5%), watch metrics/errors, then gradually increase to 25% → 50% → 100%. If something looks wrong, stop and roll back before everyone is affected.



- **Used for:** large-scale consumer apps (Netflix, Amazon-style), feature-flag driven releases.
- **Tools that help:** Kubernetes + Istio/Linkerd service mesh, Argo Rollouts, Azure App Service deployment slots with % traffic routing.

16.6 Which Strategy Do Real Companies Use?

Strategy	Downtime	Rollback Speed	Infra Cost	Typical Real-World Use
Recreate	Yes	Slow (redploy old)	Low	Dev/QA, internal tools

Rolling	No (brief mixed versions)	Medium	Low-Medium	Most Kubernetes microservices (default)
Blue-Green	No	Instant	High (2x env)	Critical prod apps, payment systems
Canary	No	Fast (stop rollout)	Medium	Large user-base products, risky features

Real scenario: "We deployed a Spring Boot pricing-engine change with Rolling strategy in Kubernetes on a Friday, and a subtle bug only showed up under load. Because it was Rolling, only some pods had the bad version, so `kubectl rollout undo deployment/pricing-service` instantly reverted us to the last stable ReplicaSet in under a minute — no manual redeploy needed."

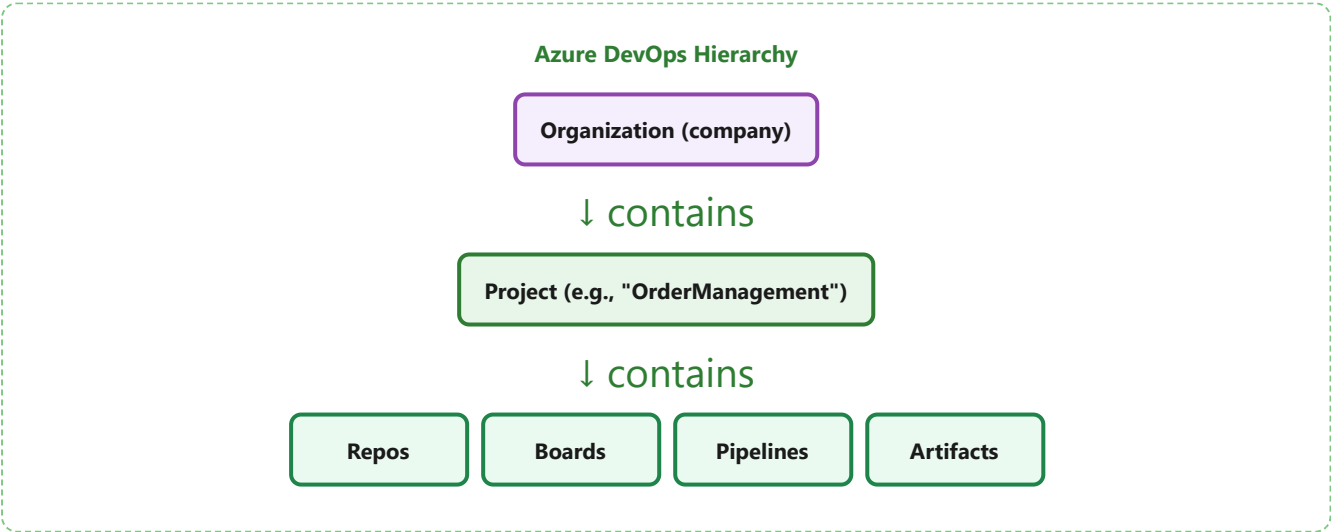
17. Azure DevOps in Depth — Our Organization's Real Setup & Configuration

17.1 What Azure DevOps Actually Is

Azure DevOps is not just "Azure Pipelines" — it is a full suite. Most organizations use these modules together:

Module	Purpose
Azure Boards	Work items, sprints, backlogs, Kanban boards (Jira-like)
Azure Repos	Git repositories (private, hosted by Microsoft) with PR reviews and branch policies
Azure Pipelines	CI/CD — build and release pipelines (YAML or Classic UI)
Azure Artifacts	Private package feeds — Maven/npm/NuGet packages internal to the org
Azure Test Plans	Manual + exploratory testing tracking

17.2 Organization Structure in Azure DevOps



- **Organization** — top level, e.g., `dev.azure.com/mycompany`.
- **Project** — one per product/team, e.g., "OrderManagement", "CustomerPortal".
- **Teams** — groups within a project with their own board/area path.

17.3 Setting Up a Project — Step by Step (What We Actually Do)

1. Create/Join Organization → Create Project (choose Git as version control).
2. Import or push existing repo into **Azure Repos** (or connect an external GitHub repo as the source).
3. Set **Branch Policies** on `main` / `develop`: require PR, minimum 2 reviewers, require linked work item, require build validation to pass.
4. Create **Service Connections** (Project Settings → Service Connections) — this is how the pipeline authenticates to Azure Kubernetes Service, Azure Container Registry, Azure App Service, SonarQube, etc.

5. Create **Variable Groups** (Pipelines → Library) — e.g., `dev-secrets`, `qa-secrets`, `prod-secrets`, each linked optionally to Azure Key Vault.
6. Create **Environments** (Pipelines → Environments) — `Development`, `QA`, `Staging`, `Production` — and add **Approval Checks** on Staging/Production.
7. Create the pipeline itself pointing to `azure-pipelines.yml` in the repo.

17.4 Service Connections (Critical Real-World Config)

A Service Connection is a stored, permission-scoped credential Azure Pipelines uses to talk to an external system, so raw secrets never sit in YAML.

Service Connection Type	Used For
Azure Resource Manager (ARM)	Deploying to App Service, AKS, Azure Functions, Storage
Docker Registry / ACR	Push/pull container images to Azure Container Registry
GitHub	Trigger pipelines from a GitHub-hosted repo
SonarQube/SonarCloud	Code quality gate integration
Kubernetes	kubectl-based deploys to a specific AKS cluster/namespace

17.5 Full Real Azure DevOps YAML — Spring Boot, Multi-Stage, Multi-Environment

```
# azure-pipelines.yml — our real organization pattern
trigger:
  branches:
    include: [ main, release/*, feature/* ]

pool:
  vmImage: 'ubuntu-latest'

variables:
  - group: common-settings          # variable group from Library
  - name: mavenGoal
    value: 'clean package'

stages:
- stage: Build
  displayName: 'Build & Unit Test'
  jobs:
  - job: BuildJob
    steps:
    - task: JavaToolInstaller@0
      inputs: { versionSpec: '17', jdkArchitectureOption: 'x64', jdkSourceOption: 'PreInstalled' }
    - task: Maven@4
      inputs:
        mavenPomFile: 'pom.xml'
        goals: '$(mavenGoal)'
        publishJUnitResults: true
        testResultsFiles: '**/surefire-reports/TEST-*.xml'
    - task: SonarQubePrepare@5
      inputs: { SonarQube: 'SonarQube-ServiceConnection', scannerMode: 'Other' }
    - script: mvn sonar:sonar
      displayName: 'SonarQube Analysis'
    - task: SonarQubePublish@5

- stage: Docker
  displayName: 'Build & Push Image'
  dependsOn: Build
  jobs:
  - job: DockerJob
    steps:
    - task: Docker@2
      inputs:
        containerRegistry: 'ACR-ServiceConnection'
        repository: 'order-service'
        command: 'buildAndPush'
        Dockerfile: 'Dockerfile'
        tags: '$(Build.BuildId)'

- stage: DeployDev
  displayName: 'Deploy to Dev'
  dependsOn: Docker
  jobs:
  - deployment: DeployDevJob
    environment: 'Development'          # no approval gate configured
    strategy:
      runOnce:
        deploy:
          steps:
          - task: KubernetesManifest@1
            inputs:
```

```

        action: 'deploy'
        kubernetesServiceConnection: 'AKS-Dev-ServiceConnection'
        namespace: 'dev'
        manifests: 'k8s/deployment.yml'
        containers: 'myacr.azurecr.io/order-service:${Build.BuildId}'

- stage: DeployProd
  displayName: 'Deploy to Production'
  dependsOn: DeployDev
  jobs:
  - deployment: DeployProdJob
    environment: 'Production'           # approval gate configured in UI – pauses here
    strategy:
      runOnce:
        deploy:
          steps:
          - task: KubernetesManifest@1
            inputs:
              action: 'deploy'
              kubernetesServiceConnection: 'AKS-Prod-ServiceConnection'
              namespace: 'prod'
              manifests: 'k8s/deployment.yml'
              containers: 'myacr.azurecr.io/order-service:${Build.BuildId}'

```

17.6 The **environment:** Keyword and Approvals (Configured in the UI, not YAML)

Notice the YAML does not contain the approval logic — that is intentional. In Azure DevOps, you configure **Approvals and Checks** directly on the Environment object (Pipelines → Environments → Production → Approvals and Checks → Add "Approvals"). This lets security/release managers change who approves without touching code.

- **Dev environment:** no approval — auto-deploys after build/test passes.
- **QA/Staging:** optional — sometimes a QA lead approval check.
- **Production:** mandatory approval — usually Release Manager + Team Lead, sometimes a 2-person rule.
- Other Checks available: Business hours restriction, branch control, invoke Azure Function for automated gate, required template.

17.7 Variable Groups & Key Vault Integration

```

# Library → Variable Group "prod-secrets" linked to Azure Key Vault "mycompany-kv"
# Secrets like DB_PASSWORD, JWT_SECRET are pulled at pipeline run-time, never stored in YAML/git
variables:
  - group: prod-secrets    # DB_PASSWORD, API_KEY resolved from Key Vault automatically

```

17.8 Azure Repos Branch Policy (What We Enforce)

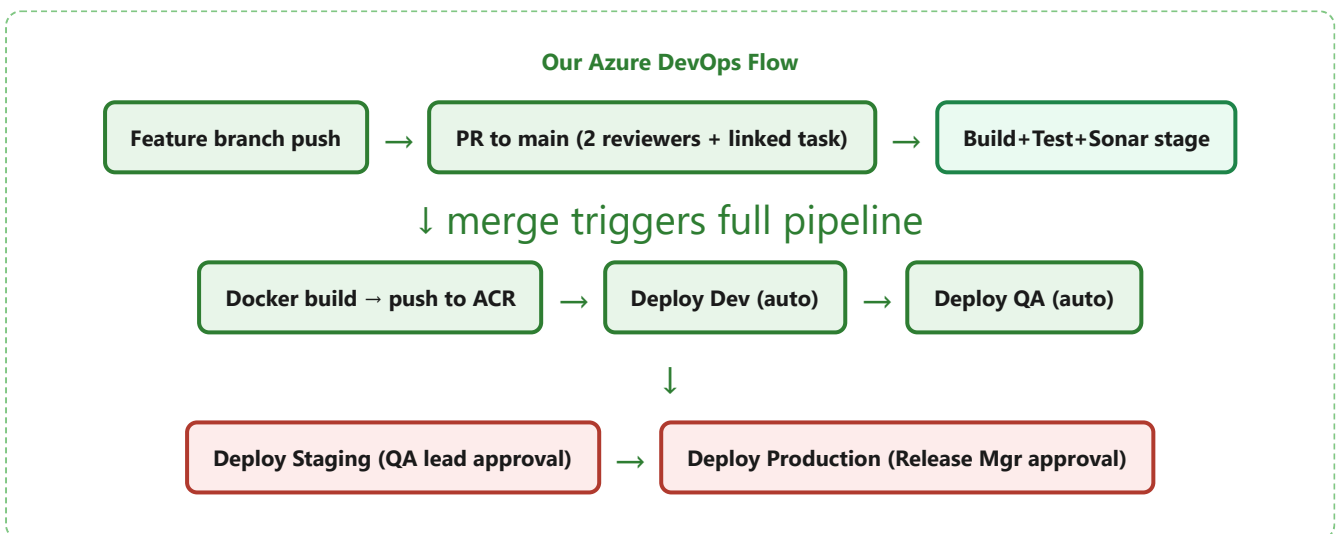
Policy	Setting We Use
Minimum reviewers	2 approvals required on main
Linked work item required	Yes — every PR must link an Azure Boards task/bug
Build validation	PR must trigger the CI pipeline and pass before merge

Comment resolution	All PR comments must be marked resolved
Merge type	Squash merge enforced (clean history)

17.9 Multi-Stage vs Classic Release Pipelines

Older Azure DevOps setups used a separate "Classic Release Pipeline" (drag-and-drop UI) after a "Classic Build Pipeline". Modern orgs (including ours) use a single **YAML multi-stage pipeline** checked into the repo — build, test, and deploy stages all in one version-controlled file, so pipeline changes go through PR review just like code.

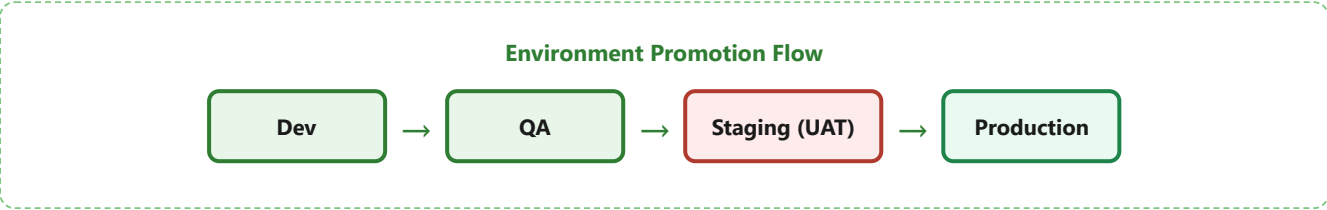
17.10 Real Organization Flow — End to End



Real scenario: "Our Production environment check also runs an Azure Function gate that verifies no P1 incidents are open before allowing the approval button to even appear — this stopped several risky Friday deployments automatically."

18. How Deployment Works (Dev → QA → Staging → Prod)

18.1 The Standard Environment Chain



Environment	Purpose	Who Uses It	Data	Approval Needed?
Dev	Developers test integration of their own changes	Developers	Fake/synthetic data	No
QA	QA team runs manual + automated test suites	QA/Testers	Test data sets	Usually no
Staging/UAT	Mirror of production for final business sign-off	Business/Product owners	Prod-like masked data	Yes (QA lead / product owner)
Production	Real users, real traffic	End customers	Real data	Yes (Release Manager)

18.2 The Same Artifact Everywhere (Key Principle)

Golden rule: Build the artifact (JAR/Docker image) **once**, then promote the exact same artifact through Dev → QA → Staging → Prod. Never rebuild code per environment — only change **configuration** (via `application-{profile}.yaml`, environment variables, or config maps). Rebuilding per environment risks "it worked in QA but broke in Prod" due to different dependency versions.

18.3 Config-Per-Environment (Spring Boot Example)

```
# same JAR, different profile activated by an environment variable at deploy time
java -jar order-service.jar --spring.profiles.active=prod

# application-dev.yml → points to dev DB, verbose logging
# application-qa.yml → points to QA DB, test payment gateway
# application-prod.yml → points to prod DB, real payment gateway, minimal logging
```

18.4 How the "Deploy" Step Physically Works

1. **Pull:** deployment target (server/AKS node) pulls the versioned image from the registry (ACR/ECR/Docker Hub).
2. **Apply config:** environment-specific variables/secrets are injected (Kubernetes ConfigMap/Secret, or App Service settings).
3. **Roll out:** old containers are replaced per the chosen strategy (Recreate/Rolling/Blue-Green/Canary — see Section 16).
4. **Health check:** orchestrator waits for `/actuator/health` (Spring Boot Actuator) to return healthy before sending traffic.
5. **Smoke test:** pipeline hits a few critical endpoints post-deploy to confirm the service is really up.

18.5 Rollback in Real Time

```
# Kubernetes instant rollback to previous ReplicaSet  
kubectl rollout undo deployment/order-service -n prod
```

```
# Azure DevOps: re-run the last known-good pipeline stage, or redeploy previous artifact version
```

19. How Integration Testing Works in a Pipeline

19.1 The Test Pyramid

Test Pyramid (Bottom = More Tests, Fastest)

E2E / UI Tests (few, slow)

Integration Tests (medium count)

Unit Tests (many, fastest)

19.2 Where Integration Tests Run in the Pipeline

- **Unit tests:** run in the Build stage, no external dependencies, mocked (JUnit + Mockito).
- **Integration tests:** run against a real (but temporary) database/queue — usually spun up as a service container in the pipeline itself using **Testcontainers** or a Docker Compose service.
- **Contract tests:** verify the frontend and backend agree on API shape (e.g., Pact) — run whenever either side changes.
- **E2E tests:** run after deployment to a real environment (QA/Staging) using Selenium/Cypress/Playwright against the deployed URL.

19.3 Real Example — Spring Boot Integration Test with Testcontainers

```
@SpringBootTest
@Testcontainers
class OrderRepositoryIntegrationTest {

    @Container
    static PostgreSQLContainer<> postgres = new PostgreSQLContainer<>("postgres:15")
        .withDatabaseName("orders_test");

    @DynamicPropertySource
    static void configureProps(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
    }

    @Autowired
    OrderRepository orderRepository;

    @Test
    void savesAndFetchesOrder() {
        Order saved = orderRepository.save(new Order("ORD-1001", 500.0));
        assertThat(orderRepository.findById(saved.getId())).isPresent();
    }
}
```

In CI, this spins up a **real, throwaway Postgres container** for the test run only — closer to production behavior than an in-memory H2 database, catching real SQL/dialect bugs early.

19.4 Pipeline Stage Definition for Integration Tests

```
- stage: IntegrationTests
  jobs:
  - job: RunIntegrationTests
    steps:
    - script: mvn verify -Pintegration-tests
      displayName: 'Run Testcontainers-based Integration Tests'
```

19.5 React Side — API Integration Testing

```
// integration test hitting a mocked backend using MSW (Mock Service Worker)
test('loads and displays orders from API', async () => {
  render(<OrderList />);
  expect(await screen.findByText('ORD-1001')).toBeInTheDocument();
});
```

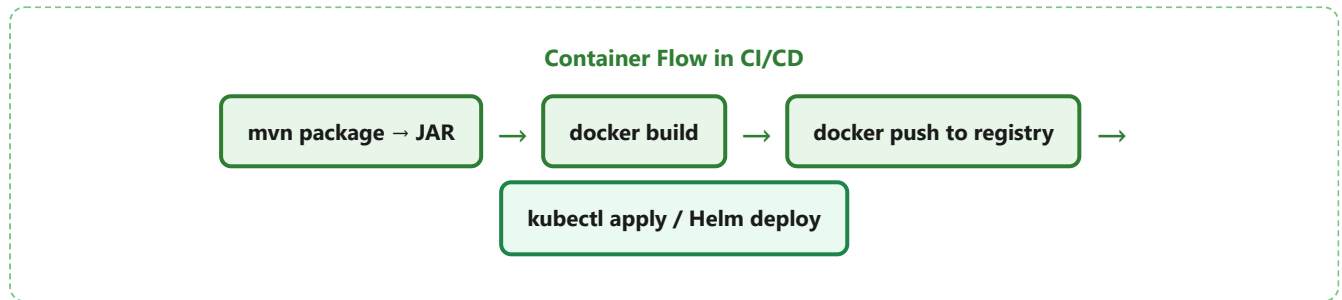
Real scenario: "Our React team uses MSW to intercept API calls in CI so frontend tests never depend on the backend being deployed — this decouples frontend and backend pipelines completely."

20. Docker + Kubernetes in the CI/CD Flow

20.1 Why Containers Fit CI/CD So Well

A container packages the app + its exact runtime dependencies into one portable unit — "works on my machine" becomes "works everywhere," because Dev, QA, Staging and Prod all run the identical image.

20.2 Where Docker Fits in the Pipeline



20.3 Kubernetes Deployment Manifest (Real Example)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
  namespace: prod
spec:
  replicas: 4
  strategy:
    type: RollingUpdate
    rollingUpdate: { maxUnavailable: 1, maxSurge: 1 }
  selector:
    matchLabels: { app: order-service }
  template:
    metadata:
      labels: { app: order-service }
    spec:
      containers:
        - name: order-service
          image: myacr.azurecr.io/order-service:BUILD_ID
          ports: [{ containerPort: 8080 }]
          readinessProbe:
            httpGet: { path: /actuator/health/readiness, port: 8080 }
            initialDelaySeconds: 10
          livenessProbe:
            httpGet: { path: /actuator/health/liveness, port: 8080 }
            initialDelaySeconds: 20
          envFrom:
            - configMapRef: { name: order-service-config }
            - secretRef: { name: order-service-secrets }
---
apiVersion: v1
kind: Service
metadata: { name: order-service, namespace: prod }
spec:
  selector: { app: order-service }
  ports: [{ port: 80, targetPort: 8080 }]
```

20.4 How the Pipeline Substitutes the Image Tag

```
# pipeline replaces BUILD_ID placeholder with the real build number, then applies
sed -i "s/BUILD_ID/${Build.BuildId}/" k8s/deployment.yml
kubectl apply -f k8s/deployment.yml -n prod
```

20.5 React Frontend in Kubernetes

React builds into static files served by Nginx inside a container — same Deployment/Service/Ingress pattern as the backend, just a much smaller, static-file image (multi-stage Docker build keeps final image tiny).

20.6 Common Kubernetes Concepts Used in CI/CD

Concept	Purpose
Deployment	Manages replica pods and rollout strategy
Service	Stable internal network endpoint for pods
Ingress	Routes external HTTP(S) traffic, TLS termination
ConfigMap	Non-secret environment configuration
Secret	Sensitive values (DB password, API keys), base64-encoded at rest
Namespace	Logical isolation per environment (dev/qa/prod namespaces in one cluster)
HPA (Horizontal Pod Autoscaler)	Auto-scales pod count based on CPU/memory/custom metrics

21. Secrets, Approvals and Environments

21.1 Why Secrets Must Never Be in Code or YAML

Committing a DB password, API key, or token to Git — even in a private repo — is a security incident. It stays in Git history forever unless the history itself is rewritten, and anyone with repo access (including former employees, compromised accounts) can read it.

21.2 Secrets Storage Options by Tool

Tool	Secrets Mechanism
Jenkins	Credentials Store (Manage Jenkins → Credentials), injected via <code>withCredentials</code>
GitHub Actions	Repo/Org/Environment Secrets — <code>\${{ secrets.NAME }}</code>
GitLab CI	CI/CD Variables (masked + protected flags)
Azure Pipelines	Variable Groups linked to Azure Key Vault, or Pipeline secret variables
AWS	AWS Secrets Manager / SSM Parameter Store, referenced by ARN in buildspec

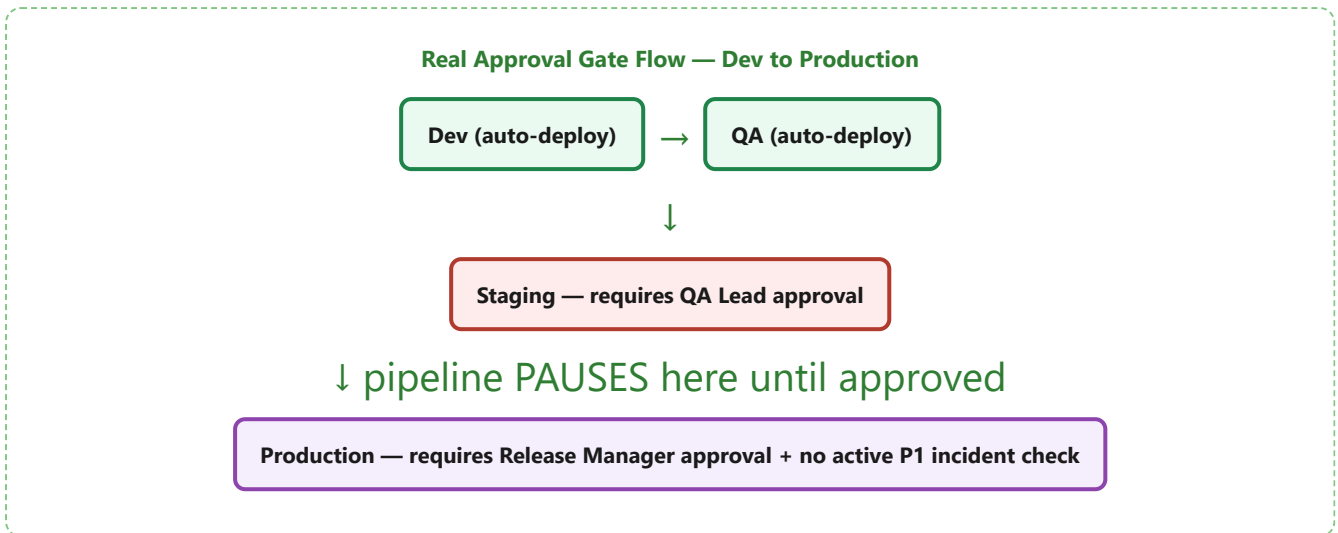
21.3 Approval Gates — Who Approves What (Real Org Pattern)

Stage	Approver	Why
Merge PR into develop/main	Peer developer / Tech Lead (code review)	Code quality, design correctness
Deploy to QA	Usually automatic	Low risk, easily reverted
Deploy to Staging/UAT	QA Lead or Product Owner	Confirms feature is ready for business validation
Deploy to Production	Release Manager / Engineering Manager (sometimes 2-person rule)	Business risk, customer impact, compliance sign-off

21.4 Environments as a First-Class Pipeline Concept

- **GitHub Actions:** `environment: production` in a job + "Required reviewers" configured on that environment.
- **Azure DevOps:** `environment: 'Production'` in YAML + Approvals/Checks configured on the Environment resource.
- **GitLab:** `environment: production` + `when: manual` job, optionally protected environment restricting who can trigger it.

Interview tip: Approvals are deliberately kept **out of YAML** in most modern tools — so a developer changing pipeline code cannot silently remove the approval requirement themselves. Approval rules live in project/environment settings which usually require admin rights to change.



22. Common CI/CD Issues and How to Fix Them (End to End Troubleshooting)

Issue	Cause	Real Fix
"Works on my machine" but pipeline build fails	Different JDK/Node version, missing env var, uncommitted local file	Pin exact tool versions in pipeline (e.g., <code>actions/setup-java@v4</code> with <code>java-version: 17</code>); use containerized/reproducible build agents
Flaky tests fail randomly in CI	Test order dependency, shared test DB state, timing/async issues	Isolate tests (Testcontainers per test class), avoid <code>Thread.sleep</code> , use explicit waits/polling
Pipeline is very slow	No caching of Maven/npm dependencies, sequential stages that could run in parallel	Cache <code>~/.m2</code> and <code>node_modules</code> , parallelize independent jobs, run only affected module tests
Secrets leaked in logs	Secret echoed by a script step, masking not applied	Use built-in secret masking, never <code>echo \$SECRET</code> , mark variables secret/protected
Deployment succeeded but app is down	No health check gating traffic cut-over	Add readiness/liveness probes; pipeline should fail if health check doesn't pass post-deploy
"Works in QA, breaks in Prod"	Different artifact built per environment, config drift	Build once, promote same artifact; externalize config via profiles/env vars
Merge conflicts breaking main branch pipeline	Long-lived feature branches, infrequent integration	Enforce short-lived branches, frequent rebase/merge from main, trunk-based development
Docker image is huge / slow to push	Not using multi-stage builds, unnecessary layers	Multi-stage Dockerfile, minimal base image (e.g., <code>eclipse-temurin:17-jre-alpine</code>)
Pipeline agent runs out of disk space	Old Docker images/build caches never cleaned	Add cleanup step, use ephemeral/hosted agents, prune Docker regularly
Rollback takes too long	Recreate strategy in prod, no versioned rollback path	Use Rolling/Blue-Green, keep last N image versions in registry, one-command rollback
Different teams' pipelines fight for the same environment	Shared Dev/QA environment, no environment locking	Namespace isolation per team/feature, ephemeral review environments per PR
SonarQube/quality gate blocks release near deadline	Technical debt piled up, coverage threshold not met	Enforce quality gates early and continuously, not just before release; treat gate failures as blocking from day one
Pipeline approval step never triggers a notification	Approvers not configured/subscribed on the Environment resource	Explicitly configure required reviewers with notification settings on each protected environment, not just access permissions
Two developers' pipelines both deploy to the same environment simultaneously, causing a race	No concurrency/lock control on the deployment stage	Enable concurrency groups / environment locks so only one deployment to a given environment runs at a time, queuing the rest
A hotfix takes as long to ship as a normal feature	No separate expedited pipeline path for urgent fixes	Maintain a lightweight hotfix pipeline/branch strategy that skips non-essential stages (e.g., long-running E2E suites) but keeps critical gates (unit tests, security scan)
Container vulnerability found only after production deployment	No image scanning step in the pipeline before deploy	Add a container scanning step (Trivy/Snyk) immediately after the Docker build stage, failing the pipeline on

		critical/high severity findings
--	--	---------------------------------

23. CI/CD Best Practices for Full Stack Teams

- **Build once, deploy everywhere** — never rebuild the artifact per environment.
- **Keep pipelines as code** — YAML/Jenkinsfile in the repo, reviewed via PR like any other code.
- **Fail fast** — put cheapest/fastest checks (lint, unit tests) first, expensive ones (E2E) last.
- **Cache dependencies** — Maven `~/.m2`, npm `node_modules`, Docker layer caching.
- **Make pipelines idempotent** — running the same stage twice should not corrupt state.
- **Automate rollback** — one command/click to revert; don't rely on manual redeploy under pressure.
- **Trunk-based / short-lived branches** — reduces merge pain and integration risk.
- **Shift security left** — SAST/dependency scanning in the pipeline, not after release.
- **Immutable infrastructure** — replace containers/instances, don't patch running ones.
- **Observability from day one** — health checks, structured logs, metrics/alerts tied to each deploy.
- **Protect main/production** — branch policies, mandatory reviews, mandatory approvals for prod.
- **Keep secrets out of code** — always use the platform's secret store / Key Vault.

24. CI/CD Cheat Sheet

Concept	One-Line Meaning
CI	Merge + build + test code automatically, frequently
CD (Delivery)	Automatically prepare a release, human approves final deploy
CD (Deployment)	Automatically deploy to production with no human step
Pipeline	The defined sequence of stages/jobs that build/test/deploy code
Stage	A logical phase (Build, Test, Deploy)
Job	A unit of work inside a stage, can run on its own agent
Runner/Agent	The machine (VM/container) that actually executes pipeline steps
Artifact	The build output (JAR, Docker image, static bundle) passed between stages
Service Connection	Stored credential letting the pipeline talk to an external system
Environment	A deployment target (Dev/QA/Staging/Prod) with its own approval rules
Rolling Deployment	Replace instances gradually, zero downtime
Blue-Green	Two full environments, instant traffic switch + instant rollback
Canary	Release to a small % of users first, expand gradually
Quality Gate	Automated pass/fail check (coverage, code smells) blocking merge/release

25. Interview Questions & Answers (Basic → Advanced → Scenario Based)

Q1. What is CI/CD in simple words?

CI (Continuous Integration) means every code change is automatically built and tested as soon as it's pushed. CD (Continuous Delivery/Deployment) means that validated code is automatically prepared for release, and optionally deployed straight to production.

Q2. Difference between Continuous Delivery and Continuous Deployment?

Continuous Delivery stops one step before production — a human clicks "deploy". Continuous Deployment removes that manual click; every passing build goes live automatically.

Q3. Why did organizations need CI/CD before it existed?

Manual releases were slow, error-prone, and involved painful "integration hell" when many developers merged code once every few weeks. CI/CD automates build/test/deploy so integration happens continuously in small, low-risk increments.

Real scenario: A team that used to do a 2-week manual release with a 6-hour deployment night now deploys 10+ times a day with zero downtime using Rolling deployments and automated pipelines.

Q4. What's the difference between Jenkins and GitHub Actions?

Jenkins is self-hosted — you install, patch, and scale the server yourself, with huge plugin flexibility. GitHub Actions is a managed service built into GitHub with YAML workflows and no infrastructure to maintain, but less control over the runner environment (unless self-hosted runners are used).

Q5. How do you handle secrets in a pipeline?

Never hardcode them. Use the platform's secret store (GitHub Secrets, Azure Key Vault via Variable Groups, GitLab CI/CD Variables, Jenkins Credentials, AWS Secrets Manager), inject as environment variables at runtime, and ensure they're masked in logs.

Q6. Explain Blue-Green vs Canary deployment.

Blue-Green keeps two full environments and switches all traffic at once — fast rollback, but costs double infra momentarily. Canary releases to a small % of real traffic first, watches metrics, then gradually ramps up — lower blast radius but slower full rollout.

Real scenario: A payment service used Blue-Green for instant rollback capability during a major schema migration, while the recommendation engine team used Canary to test a new ML model on 5% of traffic before full rollout.

Q7. What happens if a deployment fails midway in Kubernetes with Rolling strategy?

Kubernetes' `maxUnavailable` / `maxSurge` settings mean only a subset of pods are replaced at a time; if new pods fail readiness probes, the rollout pauses and old pods keep serving traffic. You then run `kubectl rollout undo` to revert to the last stable ReplicaSet.

Q8. How do you keep pipelines fast?

Cache dependencies (Maven/npm), parallelize independent jobs (frontend/backend, unit vs lint), only run affected tests when possible, use lightweight/base Docker images, and split slow E2E suites to run separately from the main gate.

Q9. What is a quality gate and why does it matter?

A quality gate (e.g., SonarQube) is an automated pass/fail rule on metrics like code coverage, duplications, and bugs/vulnerabilities. It blocks a merge or release if code quality drops below the agreed threshold, preventing technical debt from silently piling up.

Q10. Who approves deployment to production in your organization?

In our Azure DevOps setup, the Production environment has an Approvals and Checks gate — typically requiring sign-off from the Release Manager (and sometimes the Team Lead) plus an automated check confirming no active P1 incidents, before the deployment stage is allowed to run.

Q11. Explain your Spring Boot CI/CD pipeline end to end.

Push/PR triggers the pipeline → Maven compiles and runs unit + Testcontainers integration tests → SonarQube quality gate → Docker image built and pushed to ACR → deploy to Dev automatically → deploy to QA automatically → deploy to Staging with QA lead approval → deploy to Production with Release Manager approval, using Rolling deployment in Kubernetes with readiness/liveness probes.

Q12. How is a React app's pipeline different from Spring Boot's?

Instead of Maven, it uses npm/yarn for install/build/test/lint; the build output is a static bundle (not a JAR), packaged into an Nginx-based Docker image (or pushed to a static host like Azure Static Web Apps / S3+CloudFront); environment-specific API URLs are injected at build time via `.env` files or runtime config.

Q13. What's the difference between an Azure DevOps Service Connection and a Variable Group?

A Service Connection is a credential the pipeline uses to authenticate to an external system (ACR, AKS, GitHub). A Variable Group is a named collection of key/value settings (optionally linked to Key Vault) that pipelines reference for config and secrets — they solve different problems and are often used together.

Q14. How would you debug a pipeline that passes locally but fails in CI?

Check for environment differences first — JDK/Node version, timezone/locale, missing env vars, case-sensitive file paths (Linux runners vs Windows dev machine), and any local-only config/files not committed to Git. Reproduce by running the exact same commands the pipeline uses, ideally in a matching Docker container.

Q15. What is the difference between an artifact and an image in CI/CD?

An artifact is any build output passed between stages — could be a JAR, a test report, or a static bundle. A Docker image is a specific, runnable packaging of an artifact plus its runtime environment, pushed to a registry and used by the deployment step.

Q16. Why are pipeline approval rules usually kept out of the YAML file itself?

If approval logic lived in pipeline YAML, any developer able to edit that file could silently remove the approval requirement. Keeping approvals in project/environment settings (requiring admin rights to change) ensures the control can't be bypassed by a code change alone.

Q17. How do you prevent two concurrent deployments from racing against each other in the same environment?

Use the pipeline platform's concurrency control (e.g., GitHub Actions `concurrency` groups, Azure DevOps environment locks) so a second deployment to the same environment queues behind the first rather than running simultaneously and causing inconsistent state.

Q18. Where should container image vulnerability scanning happen in the pipeline, and why there?

Immediately after the Docker build stage, before the image is pushed to the registry or deployed anywhere — catching critical/high vulnerabilities at this point blocks a compromised image from ever reaching production, rather than discovering the issue after the fact.

Q19. How would you design a faster pipeline path specifically for urgent hotfixes?

Maintain a lightweight hotfix branch/pipeline that still runs essential gates (unit tests, security scan, a smoke-test deploy) but skips slower, non-essential stages like full E2E suites — balancing speed with not skipping the checks that actually prevent a bad hotfix from making things worse.

Q20. What's the real risk of not having notifications configured for pipeline approval requests?

A deployment can sit silently blocked for hours/days waiting on an approval nobody knows is pending, delaying releases and creating false alarms about "pipeline is broken" — always configure explicit reviewer notifications alongside the approval permission itself.

26. Final Summary

Before CI/CD: manual builds, manual testing, painful multi-week releases, frequent "it broke in prod" surprises.

After CI/CD: every change is automatically built, tested, quality-gated, containerized, and promoted through Dev → QA → Staging → Production with clear approval ownership, fast rollback, and full traceability from commit to deployed pod.

The 5 tools compared: Jenkins (self-hosted, maximum control), GitHub Actions (native to GitHub, huge marketplace), GitLab CI (tightly integrated with GitLab MRs), Azure Pipelines (enterprise-friendly, deep Azure integration — our current tool), AWS CI/CD (fully native AWS service chain).

Deployment strategies: Recreate (simplest, downtime), Rolling (default, zero downtime), Blue-Green (instant rollback, costly), Canary (safest for large user bases, gradual).

The non-negotiables: build once and promote the same artifact, keep secrets out of code, gate production behind real approvals, and always have a fast rollback path.

Use this guide alongside the Git/GitHub/GitLab/Bitbucket guide — together they cover the full real-world Java full-stack developer workflow from writing code to it running safely in production.